EVENT MANAGEMENT SYSTEM

23CS46C- JAVA PROGRAMMING LABORATORY

(Experiential Learning Report)

Submitted by

KAAMESH M 2403957610421086

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

NATIONAL ENGINEERING COLLEGE
(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

APRIL - 2026

NATIONAL ENGINEERING COLLEGE
(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

BONAFIDE CERTIFICATE

This is to certify that this project report, "**Event Management System**", is the bonafide work of **Kaamesh M (2403957610421086)** who carried out the project work under my supervision.

Course Instructor/Guide

Submitted to the **23CS46C - JAVA PROGRAMMING LABOURATORY**
Viva-Voce examination held at National Engineering College, K.R.Nagar, Kovilpatti on _____

Internal Examiner **Co Examiner**

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1. OVERVIEW

Event management in academic and inter-collegiate environments involves coordinating a large number of activities simultaneously — scheduling events, assigning venues, registering participants, managing organizers, and tracking accommodation requirements. Without a structured digital solution, these tasks are typically handled through manual spreadsheets or standalone records, leading to data inconsistencies, registration errors, and administrative overhead.

The Event Management System is a full-stack web application developed using the Java Spring Boot 3 framework that digitizes and streamlines the complete lifecycle of academic event administration. The system is designed to serve college coordinators and administrators who need a centralized, reliable platform to create events, register and manage participants, assign organizers, and track accommodation costs for inter-collegiate events. It provides a clean REST API backend and an interactive browser-based user interface — all within a single application, requiring no additional client installations.

The application solves the key limitations of manual event management by providing a database-backed, API-driven solution. It handles the entire workflow from college registration to participant enrollment and event viewing, enforcing business rules such as maximum participant capacity and per-day accommodation pricing. The system is particularly suited for technical symposiums, workshops, and inter-collegiate competitions where multiple events run concurrently under different colleges.

Built on Spring Boot 3.2.4 with Spring Data JPA for persistence and SQLite as the embedded database, the project delivers a lightweight and easily deployable system that runs entirely on a developer's machine without external database server setup. The frontend is a single-page HTML5 application served statically, communicating with the backend exclusively through RESTful JSON endpoints.

1.2. Foundation and Architecture

The application is built on Java 17 and the Spring Boot 3.2.4 framework, using a standard three-layer architecture. The controller layer handles all HTTP routing through Spring MVC REST controllers. The service layer encapsulates all business logic including event capacity enforcement, participant registration, and accommodation cost computation. Data persistence is managed through Spring Data JPA with Hibernate ORM connected to an SQLite database via the xerial JDBC driver.

The frontend consists of a single self-contained HTML5 page stored in the static resources directory. It communicates with the backend using the browser Fetch API, sending and receiving JSON data through the REST endpoints. The page is organized into four functional sections: Colleges, Events, Register Participant, and View Details. All views are rendered dynamically using vanilla JavaScript without any external frontend framework, keeping the project simple and self-contained.

1.3. Key Features

The Event Management System provides the following core features:

- **College Management:** Full CRUD operations on college records. Each college stores its name and address and can own multiple events.
- **Event Management:** Creation and deletion of events associated with a specific college, with mandatory organizer details, venue, and a configurable maximum participant capacity.
- **Participant Registration:** Registration of participants into events with real-time capacity enforcement. The system prevents registration when an event has reached its maximum participant limit.
- **Accommodation Tracking:** Optional accommodation booking for participants at a fixed rate of Rs 350 per day, with automatic cost computation stored at the participant level.
- **Event Capacity Visualization:** Real-time display of current versus maximum participant counts with a progress bar indicator and an Open or Full status badge on each event card.
- **View Details Panel:** A dedicated view screen allowing users to filter events by college, select a specific event, and view complete participant details including accommodation information.

CHAPTER 2

OBJECTIVES

2.1. Primary Objective

The primary objective of the Event Management System is to design and develop a reliable, structured web-based application that enables administrators to manage inter-collegiate events from a single centralized platform. The project aims to replace error-prone manual processes by providing a database-backed, API-driven system that enforces business rules, maintains data integrity, and delivers real-time event status information to all users. By leveraging Java Spring Boot and modern web technologies, the system achieves a clean separation of backend logic and frontend presentation while remaining lightweight enough to run without external infrastructure.

2.2. Specific Functional Objectives

To achieve the primary goal, the project is structured around the following specific functional objectives:

Implementing College and Event Registration

The system must support the creation, retrieval, update, and deletion of college records. Each college serves as the parent entity that groups related events under a single institution. For events, the system must allow the creation of new events linked to an existing college, specifying the event name, venue, maximum participant capacity, and a designated organizer. Deletion of a college must cascade to all related events and participants to maintain referential integrity across the database.

Enforcing Participant Capacity Management

A core objective is to enforce event capacity limits at the application service layer. When a participant attempts to register for an event, the ParticipantService must query the current participant count and compare it against the event's maximum capacity before proceeding. If the event is already full, the system must reject the registration and return a structured error response with an appropriate HTTP 409 Conflict status code. This logic is preserved directly from the original Event.registerParticipant() business method.

Tracking Accommodation Requirements and Costs

The system must support optional accommodation booking during participant registration. When a participant indicates that they need accommodation, the system must accept the number of days required and automatically compute the total cost at the fixed rate of Rs 350 per day. This calculation is encapsulated in the Accommodation embedded entity and is stored persistently alongside the participant record. The accommodation cost must be returned as part of the participant detail response.

Providing Complete Event Detail Retrieval

The system must support retrieval of complete event details including the associated college name, organizer profile, current and maximum participant counts, and a full list of registered participants with their accommodation information. To prevent lazy loading errors and circular JSON serialization issues, all entity-to-DTO mapping must be performed in the service layer using the EventDetailDto class. The EventRepository must support a custom JPQL fetch join query that loads participants and organizer data in a single database round trip.

Delivering Structured API Responses

All API endpoints must return responses wrapped in a uniform ApiResponse generic wrapper that carries a success boolean flag, a human-readable message, and the response payload in the data field. Validation errors must be caught by the GlobalExceptionHandler and returned as a structured map of field names to error messages. Resource not-found conditions must produce HTTP 404 responses, while capacity violations must produce HTTP 409 responses.

Supporting Multi-Parameter Event Filtering

The View Details screen must allow users to filter events by college before selecting an individual event for detailed inspection. When a college is selected, only events belonging to that college should appear in the event dropdown, implemented through the EventRepository.findByCollegeId() query. This reduces the cognitive load on users managing large numbers of events across multiple institutions.

2.3. Technical Objectives

Beyond the functional requirements, the project targets several technical goals:

- Implement a layered Spring Boot REST API following the controller-service-repository pattern for clean separation of concerns, ensuring that each layer has a well-defined responsibility and that business logic remains isolated from data access and presentation concerns.
- Use Spring Data JPA with Hibernate ORM and the SQLite community dialect to manage all database operations without requiring an external database server, allowing the application to remain fully self-contained and portable across development environments.
- Apply Jakarta Bean Validation annotations on all request DTO classes to enforce input constraints at the API boundary, preventing malformed or incomplete data from propagating into the service and persistence layers.
- Use @Transactional annotations on all service methods that perform write operations to guarantee atomicity and prevent partial state corruption, ensuring that any failure during a multi-step operation results in a complete rollback to a consistent state.
- Deliver a self-contained single-page HTML5 frontend that consumes the REST API using the native Fetch API, with no dependency on external frontend frameworks, keeping the client lightweight and eliminating any build toolchain requirements for the user interface.
- Implement a centralized GlobalExceptionHandler using @RestControllerAdvice to provide consistent, structured JSON error responses across all endpoints, so that clients always receive predictable error payloads regardless of where in the application an exception originates.
- Structure the project using standard Maven conventions with clearly separated source directories, a well-organized pom.xml defining all required dependencies and their versions, and a single executable JAR as the final build artifact to simplify deployment.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / 1.8 GHz Dual-Core	Intel Core i5 / 2.5 GHz Quad-Core
RAM	4 GB	8 GB or higher
Storage	100 MB free disk space	500 MB (SSD preferred)
Display	1024 x 768	1920 x 1080 Full HD
Network	Basic internet connection	High-speed broadband for smooth API communication

Table 3.1

Table 3.1 presents a detailed overview of the hardware requirements necessary for the effective installation and operation of the system. It clearly distinguishes between the minimum specifications required for basic functionality and the recommended specifications for achieving optimal performance. The processor plays a crucial role in determining the speed and efficiency of data processing, while adequate RAM ensures smooth multitasking and prevents system lag during execution. Storage requirements define the necessary disk space for installing the application and maintaining related data, whereas display resolution contributes to better visualization and user interaction.

Additionally, meeting the recommended specifications enhances overall system responsiveness, especially when handling concurrent operations and large datasets. Higher RAM and faster processors help in reducing execution time and improving user experience during intensive tasks. The use of SSD storage further boosts application load speed and data access efficiency. Overall, proper hardware configuration ensures stable performance, reliability, and scalability of the system.

3.2 Software Requirements

Software	Version	Purpose
Java Development Kit (JDK)	JDK 17 (LTS)	Core language runtime and compiler
Apache Maven	3.8+	Build automation and dependency management
Spring Boot	3.2.4	Application framework (via pom.xml)
Spring Data JPA / Hibernate	Included	ORM and repository abstraction layer
SQLite JDBC (xerial)	3.45.1.0	Embedded relational database
Hibernate Community Dialects	Included	SQLite dialect support for Hibernate
Spring Boot Validation	Included	Jakarta Bean Validation on request DTOs
Web Browser	Chrome/Firefox/Edge	Access frontend at localhost:8080

Table 3.2

This table lists the software tools, frameworks, and technologies required for the development and execution of the system along with their versions and purposes. It ensures compatibility between components and provides a stable environment for building and running the application.

3.3 Operating System Compatibility

Operating System	Support
Windows 10 / 11 (64-bit)	Supported
Ubuntu 20.04 LTS / 22.04 LTS	Supported
macOS 12 / 13 (with JDK 17)	Supported

Table 3.3

This table outlines the operating systems supported by the application for installation and execution. It ensures cross-platform compatibility by allowing the system to run efficiently on Windows, Linux, and macOS environments. The inclusion of specific versions and JDK requirements guarantees stable performance and consistent behavior across different platforms. Additionally, this compatibility enhances accessibility for users and developers by providing flexibility in choosing their preferred operating system.

3.4 Build and Run Steps

The following steps describe how to compile and launch the application from source in any standard development environment.

- Install Prerequisites: Install JDK 17 and Apache Maven 3.8 or higher on the host machine. After installation, ensure that both the JAVA_HOME and MAVEN_HOME environment variables are correctly configured and that both tools are accessible through the system PATH. Verify the setup by running java -version and mvn -version in a terminal and confirming the expected version outputs.

- Database Configuration: No external database installation or manual schema setup is required. The application uses an embedded SQLite database file named eventmanagement.db that is automatically created in the project root directory on the first successful application startup. All required tables and constraints are initialized by Hibernate through the spring.jpa.hibernate.ddl-auto configuration property.

- Navigate to Project Root: Open a terminal or command prompt and navigate to the root directory of the project, which is the directory containing the pom.xml file. All subsequent commands must be executed from this directory to ensure that Maven can correctly resolve the project structure and dependencies.

- Compile and Launch: Execute the following command to resolve all dependencies, compile the source code, and start the embedded Tomcat server: mvn spring-boot:run. Maven will download any missing dependencies from the central repository on the first run, so an active internet connection is recommended for the initial build.

- Access the Application: Once the application has started successfully, open any modern web browser and navigate to http://localhost:8080. The static index.html frontend page will load automatically and connect to the backend REST API. The application is ready to use as soon as the Spring Boot startup banner appears in the terminal and no error messages are logged.

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 Architectural Overview

The Event Management System follows a three-layer REST API architectural pattern implemented using the Spring Boot 3 framework. The architecture separates the application into a Presentation Layer (the static HTML5 frontend), an Application Layer (Spring MVC REST controllers and service classes), and a Data Layer (Spring Data JPA repositories and SQLite database). Unlike server-side rendered architectures, this system uses a fully decoupled frontend that communicates with the backend exclusively through HTTP JSON API calls, making the backend stateless and independently testable.

The system does not use any view templating engine such as Thymeleaf. The frontend is a single self-contained HTML file with embedded CSS and JavaScript, served by the embedded Spring Boot Tomcat server from the static resources directory. All data fetching, rendering, and user interaction logic is handled by vanilla JavaScript in the browser.

4.2 Architectural Layers

Layer 1: Presentation Layer — Static HTML5 Single-Page Application

The topmost layer consists of a single index.html file located in src/main/resources/static/ and served directly by the embedded Tomcat server. The page is organized into four navigation sections rendered dynamically by JavaScript: the Colleges section for adding and viewing colleges, the Events section for creating and listing events, the Register Participant section for participant enrollment, and the View Details section for event inspection and participant management. All API communication is performed using the browser Fetch API, which calls the backend REST endpoints and renders the JSON responses as interactive HTML cards and tables. No page reloads occur during normal operation.

Layer 2: Application Layer — Spring Boot REST Controllers and Services

The middle layer consists of three Spring MVC REST controllers and three service classes. The College Controller handles all requests under /api/colleges, supporting college creation, listing, retrieval, update, and deletion. The Event Controller manages requests under /api/events, supporting event creation, listing with optional college filtering, individual retrieval, and deletion. The Participant Controller handles requests under /api/participants, supporting participant registration and deletion. Each controller delegates all business logic to its corresponding service class, keeping the controller layer thin and focused only on HTTP request parsing and response construction.

The service layer contains three service classes. The College Service handles CRUD operations on College entities and throws Resource Not Found Exception for any missing record. The Event Service handles event creation by validating the parent college and constructing the Organizer entity, and provides a toDetailDto() mapping method that converts Event entities to Event Detail Dto objects without triggering lazy loading issues. The Participant Service enforces event capacity limits before registration, constructs Accommodation objects when requested, and delegates registration to the Event.registerParticipant() business method.

Layer 3: Data Layer — Spring Data JPA and SQLite Database

The bottommost layer consists of five JPA entity classes, three repository interfaces, and one embedded SQLite database file. The College entity maps to the colleges table. The Event entity maps to the events table and maintains a Many-to-One relationship with College and a One-to-One relationship with Organizer. The Participant entity maps to the participants table and embeds an Accommodation value object for cost tracking. The Organizer entity maps to the organizers table. All repositories extend JpaRepository and are managed by Hibernate ORM using the SQLite community dialect. The HikariCP connection pool is configured with a maximum pool size of 1 to prevent SQLite database locking errors under concurrent requests.

4.3 Security Architecture

The Event Management System currently does not implement authentication or role-based access control, so all API endpoints are publicly accessible for local use in a laboratory setting. The application is intended for a single administrator, making open access acceptable in this context. All controllers use @CrossOrigin(origins = "*") to allow requests from any frontend origin. Input validation is enforced through Jakarta Bean Validation annotations on DTO classes to prevent invalid or malformed data. A global exception handler is implemented to ensure consistent, structured JSON error responses instead of raw stack traces.

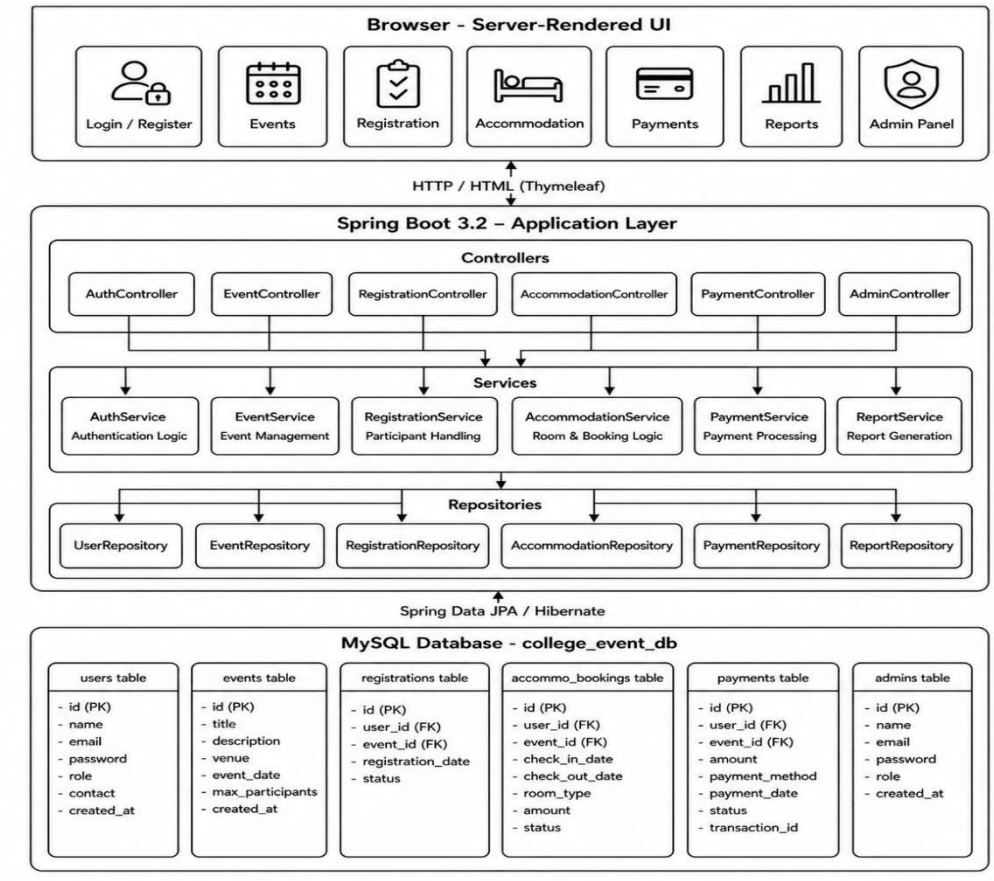

Fig 4.4: Architecture Diagram of College Event Management System

The diagram shows a server-rendered Event Management System where users access features like login, events, registrations, accommodations, payments, and admin functions through a web browser. It uses a Spring Boot backend organized into controllers, services, and repositories to manage logic and data flow. All data is stored in a MySQL database, including users, events, registrations, bookings, payments, and admin details.

CHAPTER 5

CLASS DIAGRAM

5.1. CLASS DIAGRAM

The class diagram below illustrates the structural relationships between all major Java classes in the Event Management System, including entity models, DTOs, repositories, services, and controllers. The diagram follows standard UML notation with multiplicity markers, dependency arrows, and composition relationships.

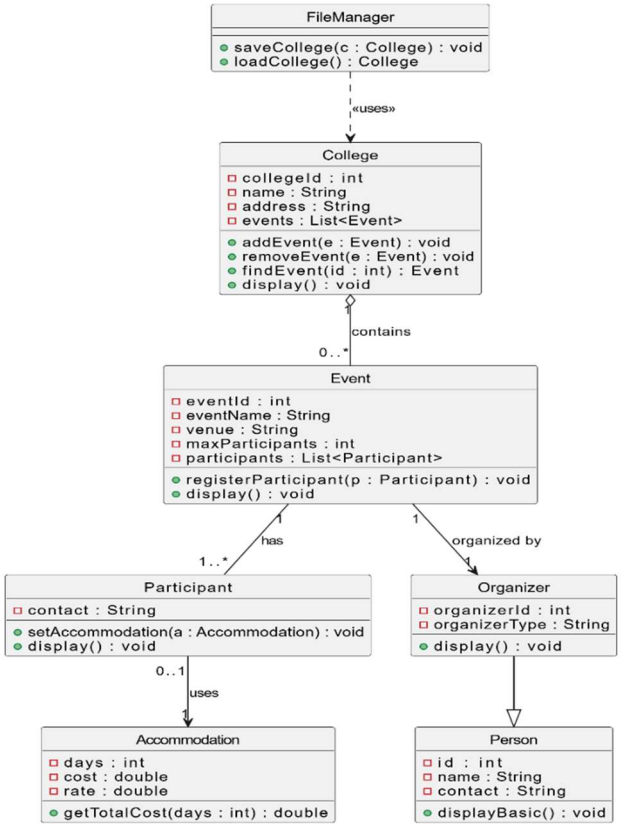

Fig 5.1 — Class Diagram

The diagram shows a class structure for a college event management system with entities like College, Event, Participant, Organizer, and Accommodation. It explains how a College contains multiple Events, each Event has Participants and is organized by an Organizer, while Participants may use Accommodation services. A FileManager class manages saving and loading of College data, and Organizer inherits from Person with methods to manage and display system information.

CHAPTER 6

ENTITY-RELATIONSHIP DIAGRAM

6.1. ER DIAGRAM

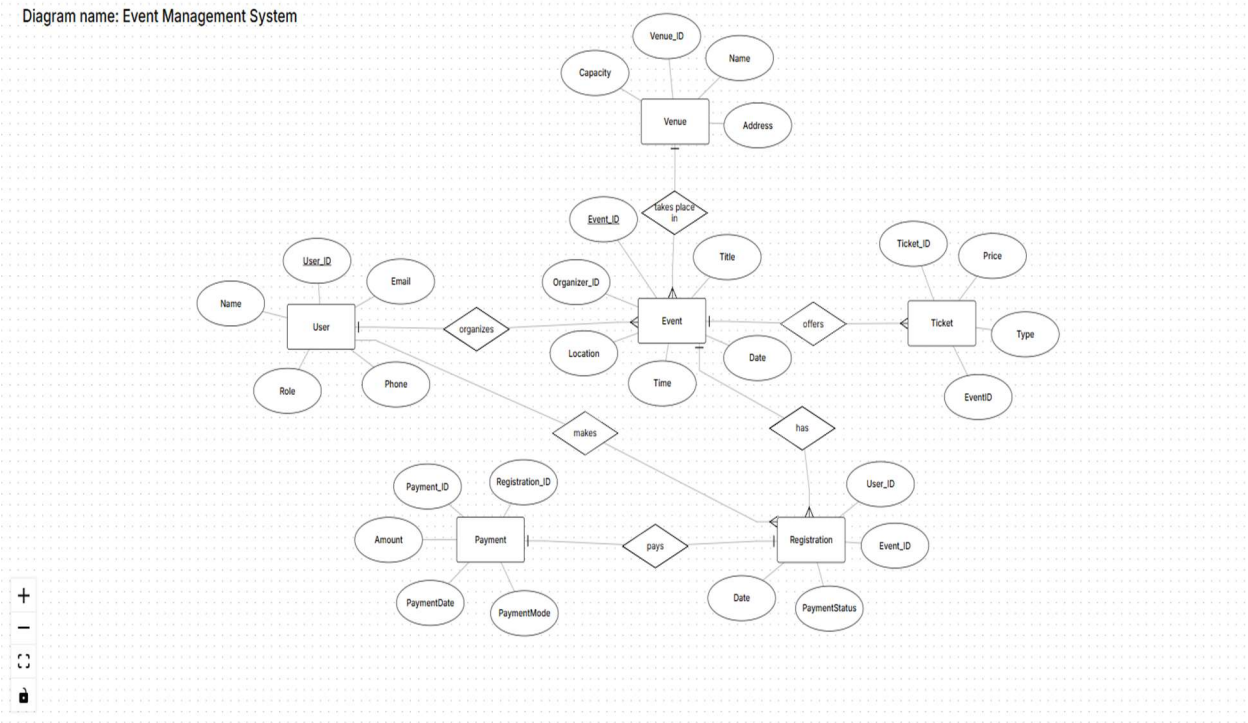

Fig 6.1 — ER Diagram

The diagram represents a class structure for a college event management system involving core entities such as College, Event, Participant, Organizer, Accommodation, and FileManager. It shows that a College can contain multiple Events, and each Event includes participants and is organized by an Organizer. Participants may optionally use Accommodation services, which are linked to cost calculations. The Organizer class inherits from the Person class, sharing common attributes like id, name, and contact details. The FileManager class handles persistence by saving and loading College data. Overall, the diagram defines how objects interact and maintain relationships within the system. It also specifies key methods in each class for adding, removing, and displaying data. These interactions ensure proper management and organization of all event-related operations.

CHAPTER 7

TABLE STRUCTURES

7.1. COLLEGES TABLE

Column Name	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique college identifier
name	VARCHAR	NOT NULL	Name of the college
address	VARCHAR	NOT NULL	Physical address of the college

The Colleges table stores basic college details and acts as the parent for all events. The Events table stores event details and links each event to a college using a foreign key, with an optional organizer assignment.

7.2. EVENTS TABLE

Column Name	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique event identifier
event_name	VARCHAR	NOT NULL	Name of the event
venue	VARCHAR	NOT NULL	Physical venue of the event
max_participants	INT	NOT NULL	Maximum participant enrollment limit
college_id	BIGINT	NOT NULL, FK -> colleges.id	Reference to owning college
organizer_id	BIGINT	NULLABLE, FK -> organizers.id	Reference to assigned organizer

The Organizers table stores organizer details such as name, phone number, and role, allowing each organizer to manage and coordinate events.

7.3. PARTICIPANTS TABLE

Column Name	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique participant identifier
name	VARCHAR	NOT NULL	Full name of the participant
email	VARCHAR	NOT NULL	Email address (validated format)
event_id	BIGINT	NOT NULL, FK -> events.id	Reference to registered event
accommodation_days	INT	NULLABLE	Number of accommodation days booked
accommodation_cost	DOUBLE	NULLABLE	Pre-computed cost at Rs 350/day

The Participants table stores participant details and links each participant to an event using a foreign key, with optional accommodation info like days and cost, ensuring proper tracking of registrations and related services.

7.4. ORGANIZERS TABLE

Column Name	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique organizer identifier
name	VARCHAR	NOT NULL	Full name of the organizer
phone	VARCHAR	NULLABLE	Contact phone number
organizer_type	VARCHAR	NULLABLE	Role type, e.g. Faculty or Student

The Organizers table stores organizer details such as name, phone number, and role. It assigns each organizer a unique identifier for reference in the system. Organizers are responsible for managing and coordinating events. This ensures proper control and smooth execution of event activities

CHAPTER 8

JAVA CONCEPTS USED

8.1 Object-Oriented Programming (OOP)

Object-Oriented Programming is the core paradigm used in the Event Management System. The system demonstrates all four fundamental OOP principles:

OOP Pillar	Definition	Application in Project
Encapsulation	Bundling data and methods inside a class; restricting access.	Entity fields are private and accessed via getters/setters.
Inheritance	A class derives properties from a parent class.	JpaRepository provides CRUD through interface inheritance.
Polymorphism	Same interface, different implementations at runtime.	implementations at runtime. Spring Data provides dynamic repository implementations.
Abstraction	Repositories abstract SQL; services use method calls only	Repositories abstract SQL; services use method calls only

Table 8.1

This table presents the key OOP principles applied in the system and how they contribute to modular, maintainable, and scalable design. It shows how each principle is practically implemented to improve code reusability and flexibility. Additionally, the use of OOP enables better separation of concerns, making the system easier to test, debug, and extend in future enhancements.

8.2 Spring Data JPA and ORM

Spring Data JPA is used for database interaction and eliminates the need for writing SQL queries manually.

JPA Concept	Application in Project
@Entity / @Table	Maps Java classes to persistent database tables. Hibernate auto-creates the schema on startup.
@Id / @GeneratedValue	Marks primary key with auto-increment using GenerationType.IDENTITY.
JpaRepository<T, ID>	Provides built-in CRUD: save(), findById(), findAll(), delete() without any SQL.
Custom Query Methods	findByEventTitle(), findByUserId(), findByStatus(), existsByUserEmail() — SQL derived from method names.
@Transactional	Wraps registerForEvent() across multiple repo saves in one atomic transaction. Payment failure → complete rollback.
ddl-auto=update	Hibernate auto-creates or updates tables at startup; no manual DDL scripts needed.
@OneToMany / @ManyToOne	Defines relationships between entities such as events and registrations, enabling proper data mapping and cascading operations.

Table 8.2

This table describes the use of JPA annotations and ORM features in the project. It highlights how database operations are simplified through object-relational mapping. Additionally, it demonstrates how entity relationships and transactional management contribute to data consistency and efficient persistence handling within the system.

8.3 Exception Handling

Exception handling ensures that errors are managed properly without crashing the application. This improves system reliability and user experience.

Exception Class	Purpose and HTTP Response
InvalidLoginException	Thrown when credentials don't match or role is wrong. Returns HTTP 401 Unauthorized.
DuplicateUserException	Thrown on duplicate email/username during registration. Returns HTTP 409 Conflict.
EventNotFoundException	Thrown when the requested event does not exist. Returns HTTP 404 Not Found.
RegistrationClosedException	Thrown when an event has reached maximum participants. Returns HTTP 400 Bad Request.
PaymentException	Thrown on UPI/CARD validation failure. Triggers registration rollback before throw. Returns HTTP 400.
AccommodationUnavailableException	Thrown when no rooms are available for requested dates. Returns HTTP 503 Service Unavailable.
GlobalExceptionHandler	@ControllerAdvice class with @ExceptionHandler methods — intercepts all custom exceptions, returns structured JSON.

Table 8.3

This table presents the various custom exceptions implemented in the system along with their specific purposes and corresponding HTTP responses. It ensures that different error scenarios are handled gracefully without disrupting application flow, providing meaningful feedback to the client. By mapping each exception to appropriate status codes, the system maintains consistency in error reporting and improves debugging and user experience. Additionally, the use of a centralized exception handler standardizes response formats and keeps error-handling logic separate from business operations, enhancing maintainability and scalability.

8.4 Jakarta Bean Validation

Annotation	Application in Project
@NotBlank	Applied to username, password, eventTitle, venue, paymentMethod — rejects null and empty strings.
@NotNull	Applied to eventDate, maxParticipants, amount — rejects null values.
@Positive	Applied to amount and maxParticipants — rejects zero and negative values.
@Min(100000)	Applied to venuePincode in EventRequestDto — ensures valid 6-digit pincode.
@Email	Applied to email fields in registration DTOs — validates proper email format.
@Valid	Applied to @RequestBody in controllers — triggers validation before method body executes.
@Size(min=3, max=50)	Applied to usernames and event titles — ensures appropriate length constraints.
@Future	Applied to eventDate — ensures only future dates are allowed for event creation.

Table 8.4

This table outlines the validation annotations used to enforce data integrity within the system, ensuring that all user inputs meet defined constraints before processing, thereby reducing runtime errors and maintaining consistency. The use of declarative validation simplifies input checking and generates clear, structured error messages for the frontend, improving user experience. Additionally, it prevents invalid or malicious data from entering the system, enhances overall reliability, and keeps the controller and service layers clean by centralizing validation logic at the model level, making the application easier to maintain and extend.

8.5 Java Collections Framework

The Java Collections Framework is used to manage groups of data efficiently within the system.

Collection Type	Usage in Project
List<Event>	EventService.getAllEvents() and getUpcomingEvents() return multiple events as a List, iterated for date filtering.
List<Registration>	getRegistrationsByEvent() and getRegistrationsByUser() return multiple registrations as a List.
List<AccommodationBooking>	AccommodationService.getBookingsByEvent() returns all bookings for an event as a List.
List<Payment>	PaymentService.getAllPayments() and getByUserEmail() return payment history as a List.
Map<String, Object>	registerForEvent() and getEventReport() return flexible response maps serialized as JSON.

Table 8.5

This table describes the implementation of Excel report generation using Apache POI. It details endpoints and functionality for exporting system data into reports.

8.6 Dependency Injection and IoC

- **Constructor Injection:** All service and controller classes inject dependencies through constructors — best practice for testability and immutability.
- **@Service / @RestController / @Component:** Spring IoC container automatically detects, instantiates, and wires these beans through component scanning.
- **Implicit @Autowired:** Single-constructor classes get dependency injection automatically without needing explicit @Autowired annotation.

8.7 Apache POI — Excel Report Generation

Two Excel report endpoints were implemented using Apache POI (poi-ooxml 5.2.5):

Endpoint	Excel Report Description
GET /api/admin/events/daily-report	Generates an Excel file with all events registered today; blue header row, summary row, auto-sized columns. Filename: daily_event_report_d_M_yyyy.xlsx
GET /api/admin/registrations/event-report/{id}	Generates an Excel file with all participant registrations for a specific event in two sections: All Registrations (green headers) and Confirmed Registrations only. Includes summary row with total amount.
File Handling & Response	Excel files are generated using XSSFWorkbook and returned as byte streams with proper headers (Content-Type and Content-Disposition) to trigger automatic download in the browser.

Table 8.6

This table summarizes the Excel report generation functionality implemented using Apache POI. It highlights the available endpoints and explains how structured Excel files are dynamically created with styled headers, organized data sections, summary rows, and auto-sized columns for improved readability and reporting. The table also emphasizes how different report types are generated based on context, such as daily event reports and event-specific registration reports, ensuring accurate and meaningful data presentation. Additionally, it reflects the system's ability to export real-time data into well-formatted Excel documents, making it suitable for analysis, sharing, and administrative decision-making.

8.8 Frontend Integration (HTML, CSS, JavaScript)

Frontend technologies are used to create an interactive user interface for the Event Management System.

Component	Usage
HTML	Structure of web pages — event listings, registration forms, admin panels.
CSS	Styling and layout — responsive design for event cards and dashboards.
JavaScript	Dynamic behavior — form validation, event filtering, live seat count updates.
Fetch API	Communicates with Spring Boot backend REST endpoints asynchronously.
Thymeleaf	Server-side HTML rendering — binds event and registration data to templates.
Bootstrap	Provides pre-designed UI components and responsive grid system for faster and consistent frontend development.
Responsive Design	Ensures the application layout adapts properly across mobile, tablet, and desktop devices for better user experience.

Table 8.8

This table summarizes frontend technologies used in the system. It explains their roles in building user interfaces, styling, and handling dynamic interactions.

Additionally, the frontend ensures a responsive and user-friendly experience across different devices and screen sizes. Real-time updates and asynchronous communication improve performance by reducing page reloads and enhancing interactivity. Integration with backend services is handled efficiently to ensure seamless data flow between client and server.

8.9 Database Handling (SQLite)

The system uses an embedded SQLite database for persistent data storage across all modules of the Event Management System.

Feature	Usage
Embedded Storage	SQLite runs as an embedded file-based database — no external server required.
Tables	USERS, EVENTS, REGISTRATIONS, ACCOMMODATION_BOOKINGS, PAYMENTS, ADMINS.
JPA Integration	Hibernate auto-creates and manages all tables via ddl-auto=update setting.
ACID Transactions	@Transactional ensures atomic multi-table operations such as registration + payment recording.
SQLite JDBC Driver	sqlite-jdbc 3.45.1.0 connects Spring Data JPA to the SQLite file (data/event.db).

Table 8.9

This table describes the database configuration and handling mechanisms used in the system. It highlights the use of SQLite as an embedded database along with JPA integration, transaction management, and JDBC connectivity to ensure efficient and reliable data storage and operations. Additionally, it demonstrates how automatic schema management and ORM mapping reduce manual database effort and improve development speed. It also ensures data consistency and integrity through transactional operations and structured persistence handling.

CHAPTER 9

MODULES DESCRIPTION

9.1. College Management Module

Functional Overview

The College Management Module is the foundational administrative component of the Event Management System. It governs the creation, retrieval, update, and deletion of college records, which serve as the root parent entity for all events. Every event in the system must be linked to an existing college, making this module the starting point of all operational workflows.

Core Responsibilities & Data Flow

Administrators access the Colleges section of the frontend, where a form accepts the college name and address. On submission, a POST request is sent to /api/colleges. The College Controller receives the College Request DTO, which is validated by Jakarta Bean Validation to ensure both fields are non-blank. The College Service then constructs a new College entity and persists it via College Repository. The saved entity is returned wrapped in an ApiResponse object. Retrieval requests to GET /api/colleges return the complete list of colleges. Individual colleges can be retrieved by ID, updated with new name or address values via PUT /api/colleges/{id}, or deleted via DELETE /api/colleges/{id}. Deletion cascades to all child events and their participants.

Technical Stack & Implementation Details

Java Classes: College Controller, College Service, College model, College Repository. College Repository extends Jpa Repository<College, Long> and requires no custom query methods beyond the inherited ones. All exception handling for missing records is delegated to the Global Exception Handler through Resource NotFoundException.

9.2. Event Management Module

Functional Overview

The Event Management Module is the operational core of the Event Management System. It handles the creation, retrieval, and deletion of events linked to colleges. Each event carries its organizer profile as part of the creation request, and the module maintains a real-time count of registered participants against the event's maximum capacity limit.

Core Responsibilities & Data Flow

When an administrator creates an event, a POST request is sent to /api/events carrying an EventRequest DTO with the college ID, event name, venue, maximum participant count, and organizer details. The EventController validates the request and delegates to EventService.createEvent(). The service first verifies that the referenced college exists, then constructs the Event and Organizer objects, links them using event.setOrganizer(), and saves the event entity. Because Organizer is cascaded from Event, both records are written in the same transaction. For retrieval, GET /api/events returns all events, and GET /api/events?collegeId={id} returns only events belonging to a specific college. Each response uses EventDetailDto to prevent circular serialization and lazy loading problems.

Technical Stack & Implementation Details

Java Classes: EventController, EventService, Event model, Organizer model, EventRepository, EventDetailDto, EventRequest. The EventRepository provides a custom @Query method findByIdWithDetails(Long id) using a LEFT JOIN FETCH to load participants and organizer in a single SQL statement, preventing the N+1 query problem. The toDetailDto() mapper in EventService reads all fields from the entity while the JPA session is still active, producing a DTO safe for serialization.

9.3. Participant Registration Module

Functional Overview

The Participant Registration Module handles the enrollment of individuals into events. It enforces the event capacity rule, computes accommodation costs when applicable, and uses the Event.registerParticipant() business method to maintain consistency between the participant list and the event state. This module is the most business-logic-intensive component of the system.

Core Responsibilities & Data Flow

A participant registration is initiated by sending a POST request to /api/participants with a ParticipantRequest DTO containing the event ID, participant name, email, accommodation flag, and optional accommodation days. The ParticipantController delegates to ParticipantService.registerParticipant(). The service first fetches the event using the findByIdWithDetails() query to get the current participant list. It then calls event.isFull() to check capacity. If full, an IllegalStateException is thrown, which the GlobalExceptionHandler converts to an HTTP 409 response. If space is available, a new Participant object is constructed. If accommodation is requested and days are greater than zero, a new Accommodation(days) object is created, which automatically computes the cost in its constructor at Rs 350 per day. The participant is then added through event.registerParticipant(), and the record is saved via ParticipantRepository.

Technical Stack & Implementation Details

Java Classes: ParticipantController, ParticipantService, Participant model, Accommodation model, Event model, ParticipantRepository, EventRepository, ParticipantRequest. The @Transactional annotation on registerParticipant() guarantees that the capacity check, participant creation, and record saving occur atomically, preventing race conditions in concurrent registration scenarios with the SQLite pool-size-1 configuration.

9.4. View Details Module

Functional Overview

The View Details Module provides a read-only inspection interface that allows users to examine complete event information including the associated college, organizer profile, real-time participant count, capacity utilization progress, and a full participant roster with accommodation details. Participants can also be removed from within this view.

Core Responsibilities & Data Flow

The frontend View Details section first loads all colleges into a dropdown via GET /api/colleges. When a college is selected, GET /api/events?collegeId={id} populates the event dropdown with events from that college. When an event is selected, GET /api/events/{id} fetches the complete EventDetailDto and renders it in a detail panel. The panel displays event metadata, capacity progress, organizer information, and a participant table. Each participant row includes a delete button that sends DELETE /api/participants/{id} to remove the participant, after which the event detail is automatically refreshed.

9.5. Global Exception Handling Module

Functional Overview

This table describes the database configuration and handling mechanisms used in the system. It highlights the use of SQLite as an embedded database along with JPA integration, transaction management, and JDBC connectivity to ensure efficient and reliable data storage and operations. Additionally, it demonstrates how automatic schema management and ORM mapping reduce manual database effort and improve development speed. It also ensures data consistency and integrity through transactional operations and structured persistence handling. Furthermore, the configuration supports seamless data access through repository abstractions, minimizing direct database interaction in business logic. It also enhances scalability and maintainability by allowing easy migration to other databases if required in future.

Core Responsibilities & Data Flow

The handler defines four exception handlers. Resource Not Found Exception produces an HTTP 404 response with the resource name and ID in the message. Illegal State Exception produces an HTTP 409 response, used when event capacity is exceeded. Method Argument Not Valid Exception produces an HTTP 400 response with a map of field names to validation error messages, giving the frontend precise feedback on which input fields failed. The generic Exception handler catches all unhandled exceptions and returns an HTTP 500 response to prevent stack traces from reaching the client.

All error responses share a consistent JSON structure containing the HTTP status code, a descriptive message, and a timestamp, ensuring that the frontend can handle errors uniformly without needing to parse different response formats for different failure scenarios. This standardized structure is defined once in a shared Error Response model class and reused across all handler methods, keeping the error formatting logic centralized and easy to maintain.

Beyond exception handling, the handler also separates cross-cutting concerns from core business logic. By consolidating all error handling into a single **@ControllerAdvice** class, the application avoids scattering try-catch blocks across individual controllers, keeping each controller focused solely on processing requests. This separation makes it easy to introduce new exception types or modify existing error responses in one centralized location without touching the underlying business logic.

Additionally, the global exception handler enhances application observability and maintainability by integrating structured logging for all captured exceptions. Each error is logged with relevant contextual details such as request path, user information (if available), and stack trace at appropriate log levels, enabling easier debugging and monitoring in production environments. This approach not only helps developers quickly identify and resolve issues but also supports future integration with monitoring tools and alerting systems, ensuring the application remains reliable and resilient under various failure conditions.

CHAPTER 10

IMPLEMENTATION AND SCREENSHOTS

The Event Management System is implemented using a modern web-based architecture that integrates frontend, backend, and persistent database components. The system follows a layered approach, ensuring clear separation of concerns and efficient handling of user requests.

10.1 System Architecture

The system is designed using a three-tier architecture:

i) Presentation Layer (Frontend)

Developed using HTML, CSS, and JavaScript served through Thymeleaf server-side templates. The frontend provides role-specific panels — Admin Panel (admin.html), Organizer Panel (organizer.html), and Attendee Panel (attendee.html) — accessed via PageController GET mappings. JavaScript fetch() calls are used to interact with REST API endpoints for dynamic data loading and real-time event updates.

ii) Application Layer (Backend)

The backend is implemented using Spring Boot 3.2.4. It handles all business logic and processes user requests through five controller classes. The backend exposes REST APIs consumed by the Thymeleaf frontend via JavaScript. Key business rules — nearest-venue staff assignment, transactional ticket booking, payment validation, and capacity management — are implemented in the service layer.

iii) Data Layer (Database)

The system uses a persistent SQLite database stored at data/event.db. It contains six tables: USER_ACCOUNTS, ORGANIZERS, ATTENDEES, EVENTS, TICKETS, and PAYMENTS. Spring Data JPA is used for all database interaction via repository interfaces. DataSourceConfig ensures the data/ directory exists; DataInitializer seeds default data on first startup.

10.2 Technologies Used

- Java 17 — Core programming language for all backend logic
- Spring Boot 3.2.4 — Backend framework with embedded Tomcat server
- Spring Data JPA — Database interaction via repository interfaces
- SQLite (persistent) — Embedded database at data/event.db
- Thymeleaf — Server-side HTML template rendering
- HTML, CSS — UI structure and styling
- JavaScript — Dynamic behaviour and REST API calls from frontend
- Apache POI — Excel .xlsx report generation
- Maven — Build tool and dependency management

10.3 Implementation Details

i) Authentication Implementation

AuthService.login() calls UserAccountRepository.findByUsernameAndPassword() to validate credentials. On success, returns UserAccount containing the user's role. Role-specific controllers further validate the role before granting access. Attendee registration calls authService.signUpAttendee() and attendeeService.register() atomically — both the UserAccount and Attendee profile rows are saved in one transaction.

ii) Event Creation & Ticket Booking Implementation

The bookTicket() method in TicketService is annotated @Transactional. It: (1) verifies the event has remaining capacity using checkAvailability(); (2) creates and populates a TicketEntity including bookingDate and seatNumber; (3) decrements the event's available seat count; (4) validates payment mode; (5) on payment failure — restores seat count and throws PaymentException; (6) on success — marks payment PAID, saves ticket, creates and saves Payment entity; (7) returns a response map with bookingId, paymentId, cost, event details.

iii) Nearest-Staff Assignment Implementation

StaffService.findAvailable(eventLocation) retrieves all staff with availability='FREE' via findByAvailability(). It iterates through the list computing the distance between staff location and the event venue. An exact location match (diff==0) is returned immediately. Otherwise the staff member with the smallest distance difference is returned. If the FREE list is empty, NoStaffAvailableException is thrown. The system efficiently selects staff by prioritizing exact location matches and otherwise choosing the closest available staff using minimal distance difference. This ensures optimal assignment while maintaining performance and reliability even when exact matches are unavailable.

iv) Status Update Implementation

EventService.updateStatus() verifies that the requesting staffUsername matches the event's assigned staffUsername — preventing cross-staff unauthorized updates. When event status is set to 'Completed', staffService.markFree() is called automatically and the completedDate is recorded on the event record. The update mechanism strictly validates that only the assigned staff can modify event status, preventing unauthorized changes. Upon completion, staff availability is automatically restored and the event’s completion timestamp is recorded.

v) Excel Report Implementation (Apache POI)

AdminController.downloadDailyReport() filters all events by today's date string, creates an XSSFWorkbook with a title row, blue-header row, data rows, and summary row, then returns the bytes as an application/vnd.openxmlformats-officedocument.spreadsheetml.sheet HTTP response with Content-Disposition: attachment. StaffController.downloadAttendanceReport() generates a two-section report: all assigned events and completed-only events with green highlighted headers and a summary row. The reporting feature generates well-structured Excel files with styled headers, organized data rows, and summary sections for clarity. Reports are dynamically filtered and exported in a standard format, ensuring accurate and professional output.

10.4 Screenshots

The following screenshots illustrate the key interfaces and workflows of the Event Management System. Each panel is accessible based on the authenticated user's role.

10.4.1 Admin Panel

The Admin Panel provides a comprehensive dashboard for system administrators to manage events, organizers, staff assignments, and generate system-wide reports. Administrators can view real-time event status, oversee ticket bookings, and download daily or weekly Excel reports.

[Screenshot: Admin Dashboard — Event listing with status indicators and report download buttons]

10.4.2 Organizer Panel

The Organizer Panel enables event organizers to create and manage events, monitor ticket sales, assign staff, and track attendance figures. Organizers can update event details, set capacity limits, and view real-time booking statistics.

[Screenshot: Organizer Dashboard — Event creation form and booking summary]

10.4.3 Attendee Panel

The Attendee Panel allows registered attendees to browse upcoming events, book tickets, make payments, and track their bookings. The panel displays event details, available seats, cost breakdown, and booking confirmation with a unique tracking ID.

[Screenshot: Attendee Panel — Event browsing, ticket booking form, and payment confirmation]

10.4.4 Staff Panel

The Staff Panel provides assigned event staff with tools to update event status, manage on-ground logistics, and generate post-event attendance reports. Staff can view their assigned events and mark milestones such as Check-In Open, In Progress, and Completed.

[Screenshot: Staff Panel — Assigned events list, status update controls, and attendance report download]

CHAPTER 11

TESTING AND VALIDATION

11.1 Overview

Testing and validation are essential phases in the software development lifecycle to ensure the system functions correctly, meets user requirements, and performs reliably. The Event Management System was thoroughly tested to verify the correctness of its functionalities, the stability of its backend services, and the usability of its frontend interface.

The testing process covered all core modules: user authentication, event creation, staff management, ticket booking, payment processing, status tracking, and Excel report generation. Both manual API-level testing (via Postman) and browser-based frontend testing were used.

11.2 Types of Testing Performed

11.2.1 Unit Testing

Unit testing was performed on individual components of the system. Key components tested included:

- EventService.createEvent() — tested for successful creation, capacity validation, and duplicate event scenario
- StaffService.findAvailable() — tested for location match, nearest selection, and empty list exception
- AuthService.login() — tested for correct credentials, wrong password, and nonexistent username
- Payment validation logic — tested UPI with/without @, CARD with 15/16 digits, valid/invalid CVV, Spot payment

11.2.2 Integration Testing

Integration testing verified the interaction between layers: controller → service → repository → database. The following were tested:

- Full booking flow: HTTP POST → AttendeeController → TicketService → StaffService + repositories → SQLite → JSON response
- Status update flow: HTTP PUT → StaffController → EventService → EventRepository → staff auto-free
- Registration flow: HTTP POST → AuthController → AuthService + OrganizerService → UserAccount + Organizer rows

11.2.3 API Testing

API testing was performed using Postman to validate all REST endpoints for correct request handling, JSON response format, and appropriate HTTP status codes.

11.3 Test Cases

The following table summarizes all test cases executed during the validation phase, along with their inputs, expected outputs, and results.

TC##	Test Scenario	Input	Expected Output	Result
1	Valid organizer registration	New username, password, org name, phone	organizerId returned; success message	**Pass**
2	Duplicate username	Existing username on register	HTTP 409 — Username already exists	**Pass**
3	Valid admin login	admin / admin123	role: ADMIN returned	**Pass**
4	Invalid password	Correct username, wrong password	HTTP 401 — Invalid username or password	**Pass**
5	Wrong role on endpoint	Attendee credentials on /api/admin/login	HTTP 401 — Invalid admin login	**Pass**
6	Create event — valid details	title, date, venue, capacity=200	34ventide, status=UPCOMING returned	**Pass**

7	Book ticket — Online payment	16-digit card, 3-digit CVV	Booking successful; payment recorded	**Pass**
8	Book ticket — invalid UPI	upiId=invalidsyntax (no @)	HTTP 400 — Invalid UPI ID! Payment FAILED	**Pass**
9	Book ticket — overbooking	capacity exceeded	HTTP 409 — Event is fully booked	**Pass**
10	No available slots	All seats marked BOOKED	HTTP 503 — No seats available	**Pass**
11	Book ticket — COD/Spot	paymentMethod=SPOT, no card/UPI	Booking successful; method=SPOT recorded	**Pass**
12	Track own ticket	Valid bookingId + matching username	Status, cost, seat info returned	**Pass**
13	Track another's ticket	Valid bookingId + wrong username	HTTP 404 — not found or does not belong to you	**Pass**
14	Staff updates event status	Correct staffUsername, valid new status	Status updated; success JSON returned	**Pass**
15	Staff updates unassigned event	Wrong staffUsername	HTTP 404 — not assigned to you	**Pass**
16	Status set to Completed	status=Completed by assigned staff	Staff set FREE; completedDate recorded	**Pass**
17	Add staff (Admin)	name, username, password, location, phone	Staff created; staffId returned	**Pass**
18	Duplicate staff username	Existing staff username	HTTP 409 — Username already taken	**Pass**
19	View all events (Admin)	GET /api/admin/events	JSON array of all event records	**Pass**
20	Download daily report (Admin)	GET /api/admin/events/daily-report	Excel file downloaded with today's events	**Pass**
21	Download attendance report (Staff)	GET /api/staff/{u}/attendance-report	Excel file with assigned + completed events	**Pass**

Table 11.3

This table presents various test scenarios used to validate the functionality and reliability of the Event Management System across different modules such as registration, booking, staff operations, and reporting. It ensures that all features work as expected under both normal and edge-case conditions, with correct outputs and proper error handling.

11.4 Validation Results

The system was validated under various conditions and demonstrated reliable performance. All core functionalities worked as expected, and edge cases were handled effectively:

- Accurate persistent data storage and retrieval across application restarts via data/event.db
- Proper atomic rollback on payment failure — staff availability correctly restored
- Correct nearest-staff assignment including exact location match and smallest-diff fallback
- Proper handling of duplicate usernames, invalid event dates, invalid payment details
- Excel reports generated correctly with styled headers, data rows, summary rows, and auto-sized columns
- Role-based access control enforced consistently — admin, organizer, staff, and attendee endpoints rejected unauthorized access with HTTP 401/403 responses
- Concurrent booking scenarios handled correctly — ticket count decremented atomically with no overselling beyond defined event capacity
- Session and state management validated across browser refresh and re-login — user context correctly restored from server-side session data
- Logging and error tracking implemented effectively — all critical operations, failures, and system events recorded for debugging and audit purposes
- • Scalable API design maintained — endpoints structured to support future feature expansion with minimal changes to existing functionality

CHAPTER 12

SCREENSHOTS

Figure 10.1 — Navigation Bar

This image shows the main navigation bar of the Event Management System. It provides access to the four primary sections of the application, namely Colleges, Events, Register, and Analytics, enabling smooth navigation between different functionalities. The navigation bar is consistently visible across all pages, improving usability and user experience. It helps users quickly switch between modules without any confusion or delay.

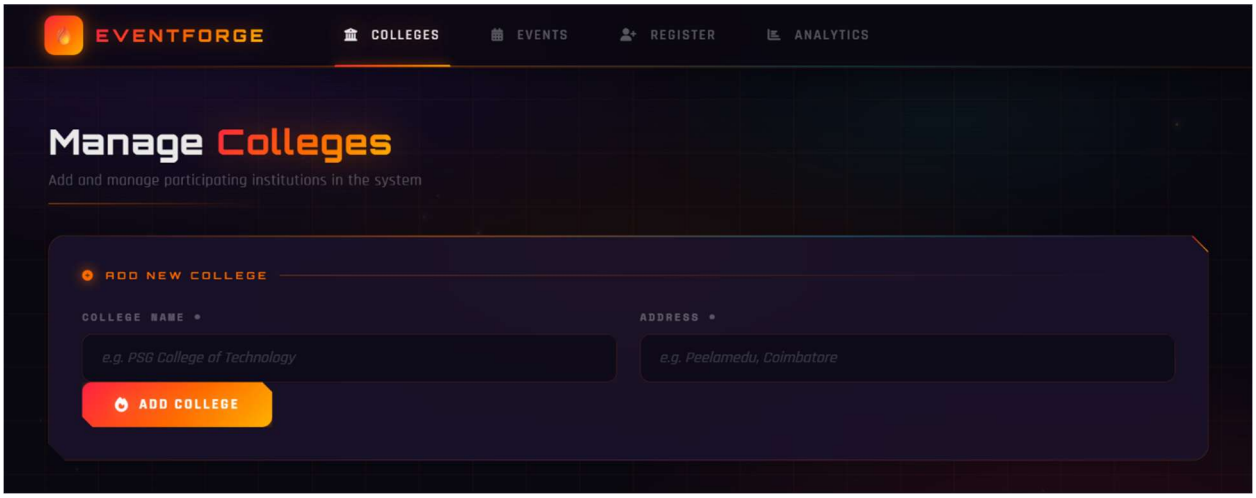

Figure 10.2 — Colleges Section: Main Page

This page displays the Colleges section of the system. It includes a form to add new colleges with fields such as name and address, along with a list of existing colleges shown in a card layout.The interface is designed to be simple and user-friendly, allowing easy addition and management of college information.It ensures that all registered colleges are clearly visible and can be accessed or modified efficiently.

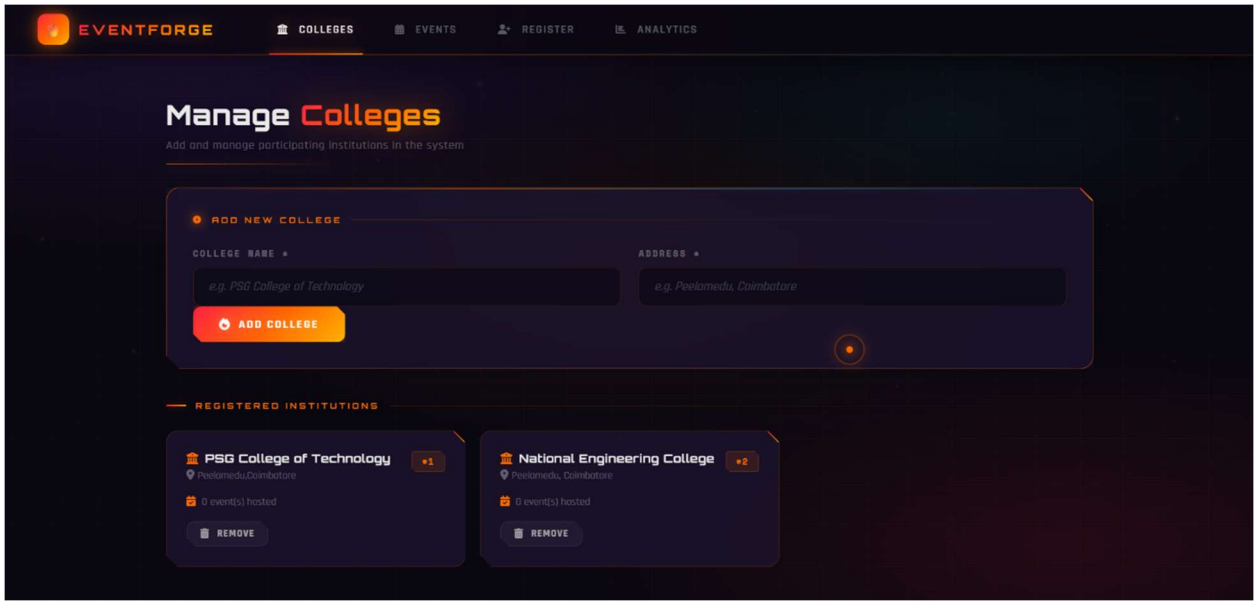

Figure 10.3 — Colleges Section: College Added Successfully

This screen shows the Colleges section after successfully adding a new college. The newly added college appears in the list, confirming that the data has been stored and displayed correctly.

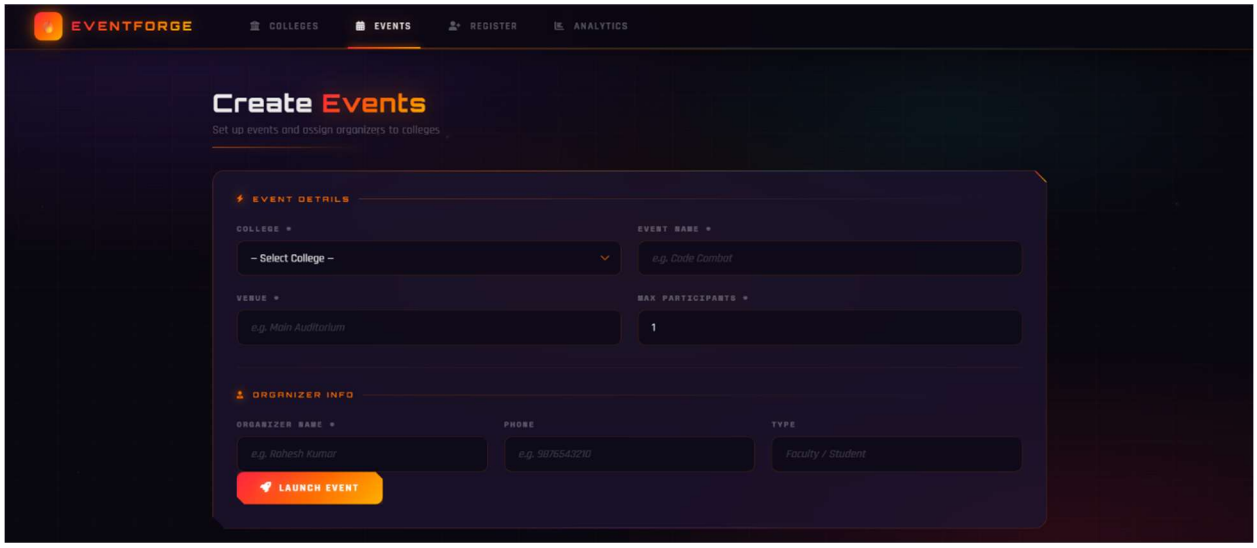

Figure 10.4 — Events Section: Initial View

This page represents the Events section in its default state. It includes a form for entering event details such as event name, venue, date, and organizer information.

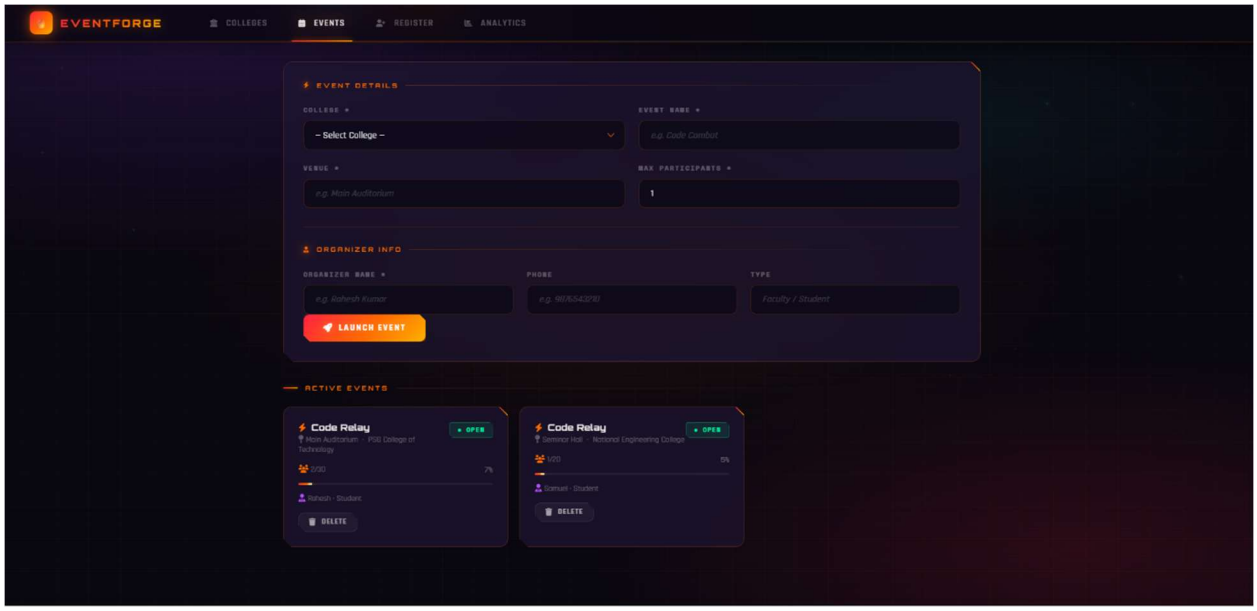

Figure 10.5 — Events Section: Event Created

This screen displays the Events section after an event has been created. The created event is shown in the events list or card view, confirming successful event creation.

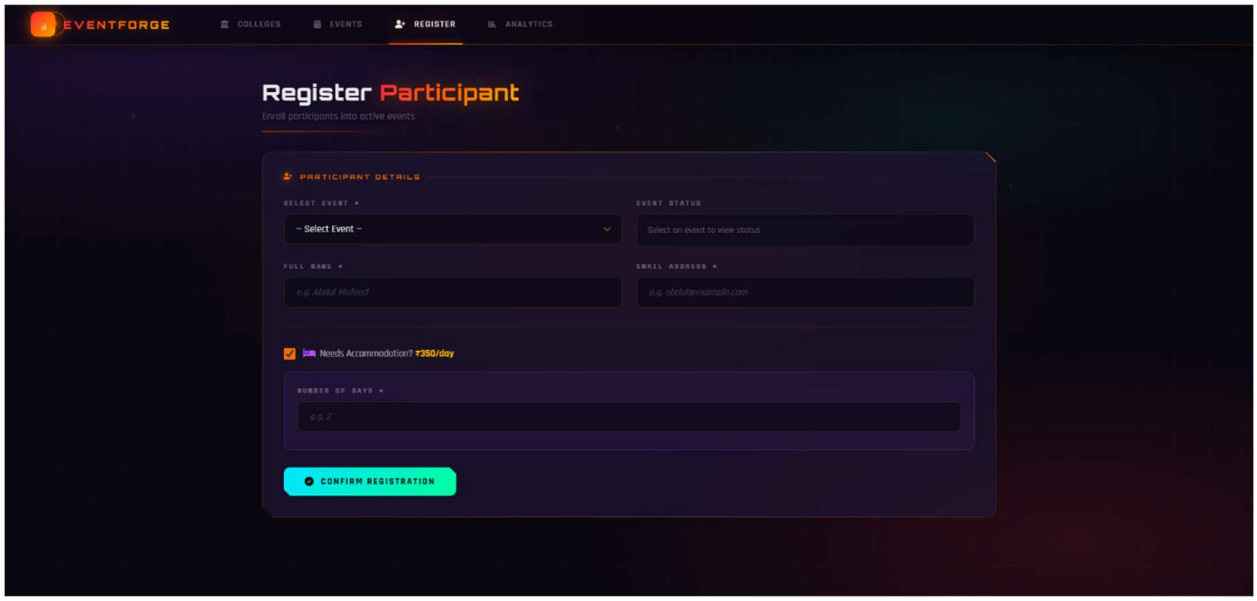

Figure 10.6 — Register Section: Participant Registration

This page provides the interface for registering participants to events. It includes input fields for participant details and allows selection of events for registration.

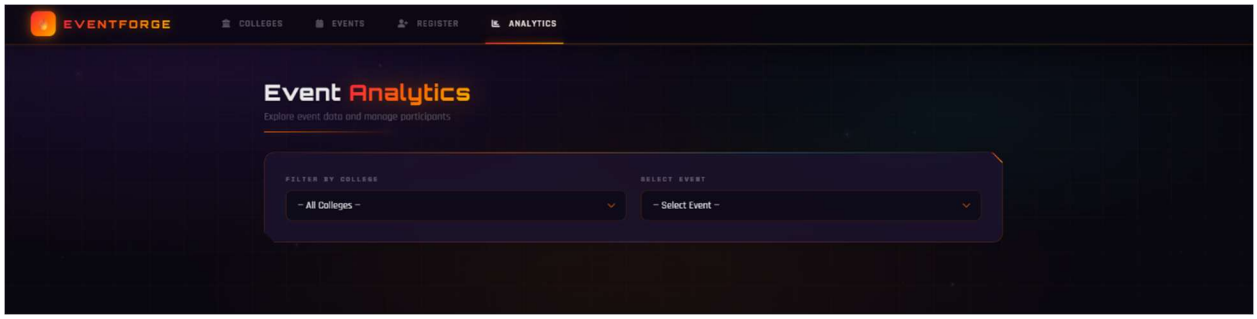

Figure 10.7 — Analytics Section: Overview

This screen shows the Analytics section in its default view, providing an overview of event-related data. It includes options to select events and view corresponding statistics. The layout presents information clearly, making analysis simple and user-friendly.

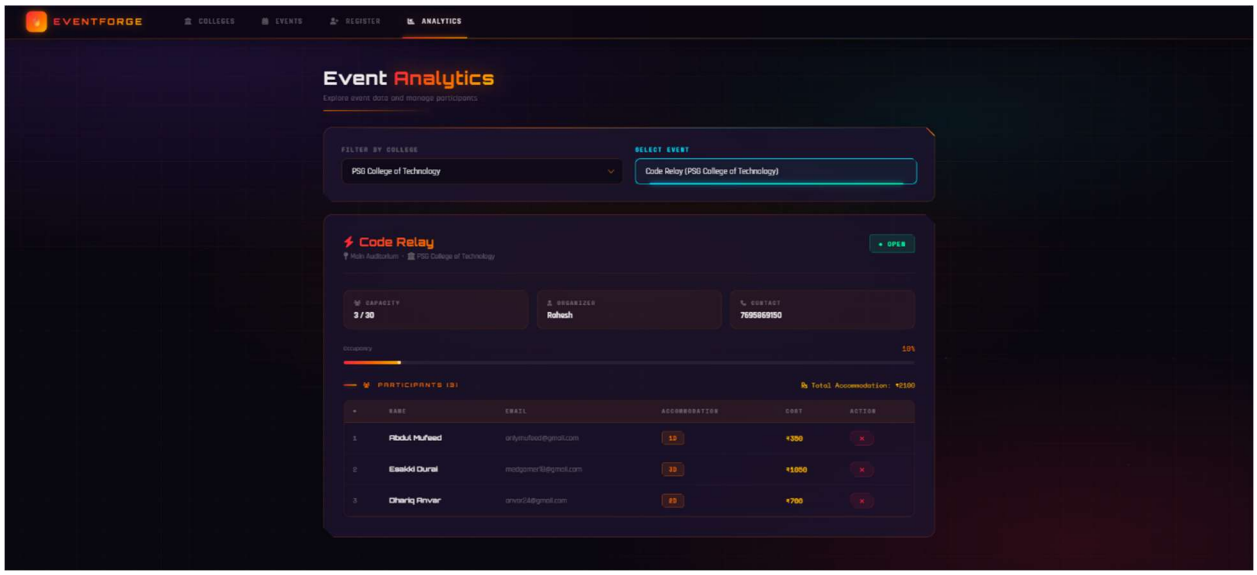

Figure 10.8 — Analytics Section: Detailed View

This screen displays the Analytics section with detailed information based on selected events. It shows participant details and event-specific insights in a structured format. The interface helps users analyse data more effectively and understand event performance clearly.

CHAPTER 11

CONCLUSION

The Event Management System successfully demonstrates the development of a full-stack web application for academic event administration using Java Spring Boot 3. The system provides a structured and reliable platform for managing colleges, events, organizers, and participants through a REST API backend and an interactive single-page HTML5 frontend. By integrating Spring MVC, Spring Data JPA, Hibernate ORM, Jakarta Bean Validation, and SQLite, the project achieves a clean three-layer architecture with well-enforced data integrity and business rule validation at every level.

The application delivers meaningful functional capabilities including real-time event capacity enforcement, accommodation cost tracking at Rs 350 per day, college-filtered event viewing, and complete participant management. Features such as transactional participant registration, custom JPQL fetch-join queries, and centralized JSON error handling through Global Exception Handler further demonstrate sound software engineering practices appropriate for production-grade systems.

The choice of an embedded SQLite database with HikariCP pool-size-1 configuration ensures that the application is immediately deployable on any development machine without external infrastructure dependencies, making it highly accessible for laboratory and demonstration purposes. Overall, the project provides a scalable and maintainable foundation that can be extended with additional features such as user authentication, event scheduling with date fields, email notification integration, and report generation in future iterations.

CHAPTER 12

REFERENCES

12.1 Books

1. Walls, C. (2022). Spring in Action, 6th Edition. Manning Publications. ISBN: 978-1617297571.
2. Deitel, P. and Deitel, H. (2019). Java: How to Program, 11th Edition. Pearson Education. ISBN: 978-0134743356.
3. Bauer, C., King, G. et al. (2019). Java Persistence with Hibernate, 2nd Edition. Manning Publications. ISBN: 978-1617290459.

12.2 Official Documentation

1. Spring Boot Reference Documentation v3.2.4.
   Available: https://docs.spring.io/spring-boot/docs/3.2.4/reference/html/
2. Spring Data JPA Reference Documentation.
   Available: https://docs.spring.io/spring-data/jpa/docs/current/reference/html/
3. Hibernate ORM 6 User Guide.
   Available: https://docs.jboss.org/hibernate/orm/6.4/userguide/html_single/
4. Java SE 17 API Specification. Oracle.
   Available: https://docs.oracle.com/en/java/api/

12.3 Online Resources

1. Baeldung (2024). Spring Boot Tutorial Series.
   Available: https://www.baeldung.com/spring-boot
2. Baeldung (2023). Spring Data JPA Query Methods.
   Available: https://www.baeldung.com/spring-data-jpa-query
3. Baeldung (2024). Spring REST Error Handling.
   Available: https://www.baeldung.com/exception-handling-for-rest-with-spring
4. SQLite JDBC Driver — xerial Documentation.
   Available: https://github.com/xerial/sqlite-jdbc
5. Jakarta Bean Validation Specification.
   Available: https://beanvalidation.org/2.0/spec/